

Parameter Learning

For each node X_u , the conditional distribution is a linear Gaussian,

$$p(x_u | x_{\pi_u}, \theta_u) = \mathcal{N}(\sum_{c \in \pi_u} w_{uc} x_c + w_u 0, \sigma_u^2)$$

Let y contain the observations of X_u , and let X contain a leading column of ones followed by the observations of its parent nodes. Then the log-likelihood is

$$\ell(w, \sigma_u^2) = -\frac{N}{2} \log(2\pi\sigma_u^2) - \frac{1}{2\sigma_u^2} \|y - Xw\|^2$$

def _learn_node_parameter_w(outputs, inputs=None):

Differentiating with respect to w and setting the result to zero gives

$$\frac{\delta \ell}{\delta w} = \frac{1}{\sigma_u^2} X^T (y - Xw) = 0$$

which gives the normal equation

$$X^T X w = X^T y$$

Hence,

$$\hat{w} = (X^T X)^{-1} X^T y$$

which can be solved directly using `np.linalg.solve()`. For nodes without parents, X consists only of the bias column.

def _learn_node_parameter_var(outputs, weights, inputs):

Differentiating with respect to σ_u^2 gives

$$\frac{\delta \ell}{\delta \sigma_u^2} = -\frac{N}{2\sigma_u^2} + \frac{\|y - X\hat{w}\|^2}{2(\sigma_u^2)^2} = 0$$

which gives the maximum-likelihood variance estimate

$$\hat{\sigma}_u^2 = \frac{1}{N} \|y - X\hat{w}\|^2$$

The implementation therefore computes the predictions and uses the mean squared residual as the variance estimate.

_get_learned_parameters(nodes, edges, observations):

For each node, I obtain its parents from the directed edges, construct the corresponding input matrix, learn its weights and variance, and store the bias, variance, and parent weights in the required dictionary. Since observations are complete, each node can be learned independently.